

TOPIC 2.1

Algorithms with Selection and Repetition

SUGGESTED SKILLS

1.A

Determine an appropriate program design to solve a problem or accomplish a task.

Required Course Content

LEARNING OBJECTIVE**2.1.A**

Represent patterns and algorithms that involve selection and repetition found in everyday life using written language or diagrams.

ESSENTIAL KNOWLEDGE**2.1.A.1**

The building blocks of algorithms include sequencing, selection, and repetition.

2.1.A.2

Algorithms can contain selection, through decision making, and repetition, via looping.

2.1.A.3

Selection occurs when a choice of how the execution of an algorithm will proceed is based on a true or false decision.

2.1.A.4

Repetition is when a process repeats itself until a desired outcome is reached.

2.1.A.5

The order in which sequencing, selection, and repetition are used contributes to the outcome of the algorithm.

SUGGESTED SKILLS

3.A

Determine the result or output based on statement execution order in an algorithm.

TOPIC 2.2

Boolean Expressions

Required Course Content

LEARNING OBJECTIVE

2.2.A

Develop code to create Boolean expressions with relational operators and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

2.2.A.1

Values can be compared using the *relational operators* `==` and `!=` to determine whether the values are the same. With primitive types, this compares the actual primitive values. With reference types, this compares the object references.

2.2.A.2

Numeric values can be compared using the relational operators `<`, `>`, `<=`, and `>=` to determine the relationship between the values.

2.2.A.3

An expression involving relational operators evaluates to a Boolean value.

TOPIC 2.3

if Statements

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

4.A

Describe the behavior of a code segment or program.

Required Course Content**LEARNING OBJECTIVE****2.3.A**

Develop code to represent branching logical processes by using selection statements and determine the result of these processes.

ESSENTIAL KNOWLEDGE**2.3.A.1**

Selection statements change the sequential execution of statements.

2.3.A.2

An `if` statement is a type of selection statement that affects the flow of control by executing different segments of code based on the value of a Boolean expression.

2.3.A.3

A *one-way selection* (`if` statement) is used when there is a segment of code to execute under a certain condition. In this case, the body is executed only when the Boolean expression is `true`.

2.3.A.4

A *two-way selection* (`if-else` statement) is used when there are two segments of code—one to be executed when the Boolean expression is `true` and another segment for when the Boolean expression is `false`. In this case, the body of the `if` is executed when the Boolean expression is `true`, and the body of the `else` is executed when the Boolean expression is `false`.

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

4.A

Describe the behavior of a code segment or program.

TOPIC 2.4

Nested `if` Statements

Required Course Content

LEARNING OBJECTIVE**2.4.A**

Develop code to represent nested branching logical processes and determine the result of these processes.

ESSENTIAL KNOWLEDGE**2.4.A.1**

Nested `if` statements consist of `if`, `if-else`, or `if-else-if` statements within `if`, `if-else`, or `if-else-if` statements.

2.4.A.2

The Boolean expression of the inner nested `if` statement is evaluated only if the Boolean expression of the outer `if` statement evaluates to `true`.

2.4.A.3

A *multiway selection* (`if-else-if`) is used when there are a series of expressions with different segments of code for each condition. Multiway selection is performed such that no more than one segment of code is executed based on the first expression that evaluates to `true`. If no expression evaluates to `true` and there is a trailing `else` statement, then the body of the `else` is executed.

TOPIC 2.5

Compound Boolean Expressions

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

Required Course Content

LEARNING OBJECTIVE**2.5.A**

Develop code to represent compound Boolean expressions and determine the result of these expressions.

ESSENTIAL KNOWLEDGE**2.5.A.1**

Logical operators `!` (not), `&&` (and), and `||` (or) are used with Boolean expressions. The expression `!a` evaluates to `true` if `a` is `false` and evaluates to `false` otherwise. The expression `a && b` evaluates to `true` if both `a` and `b` are `true` and evaluates to `false` otherwise. The expression `a || b` evaluates to `true` if `a` is `true`, `b` is `true`, or both, and evaluates to `false` otherwise. The order of precedence for evaluating logical operators is `!` (not), `&&` (and), then `||` (or). An expression involving logical operators evaluates to a Boolean value.

2.5.A.2

Short-circuit evaluation occurs when the result of a logical operation using `&&` or `||` can be determined by evaluating only the first Boolean expression. In this case, the second Boolean expression is not evaluated.

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

TOPIC 2.6

Comparing Boolean Expressions

Required Course Content

LEARNING OBJECTIVE

2.6.A

Compare equivalent Boolean expressions.

2.6.B

Develop code to compare object references using Boolean expressions and determine the result of these expressions.

ESSENTIAL KNOWLEDGE

2.6.A.1

Two Boolean expressions are *equivalent* if they evaluate to the same value in all cases. Truth tables can be used to prove Boolean expressions are equivalent.

2.6.A.2

De Morgan's law can be applied to Boolean expressions to create equivalent Boolean expressions. Under De Morgan's law, the Boolean expression $\neg(a \ \&\& \ b)$ is equivalent to $\neg a \ || \ \neg b$ and the Boolean expression $\neg(a \ || \ b)$ is equivalent to $\neg a \ \&\& \ \neg b$.

2.6.B.1

Two different variables can hold references to the same object. Object references can be compared using `==` and `!=`.

2.6.B.2

An object reference can be compared with `null`, using `==` or `!=`, to determine if the reference actually references an object.

2.6.B.3

Classes often define their own `equals` method, which can be used to specify the criteria for equivalency for two objects of the class. The equivalency of two objects is most often determined using attributes from the two objects.

X EXCLUSION STATEMENT—*Overriding the `equals` method is outside the scope of the AP Computer Science A course and exam.*

TOPIC 2.7

while Loops

SUGGESTED SKILLS**2.A**

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

Required Course Content

LEARNING OBJECTIVE**2.7.A**

Identify when an iterative process is required to achieve a desired result.

2.7.B

Develop code to represent iterative processes using `while` loops and determine the result of these processes.

ESSENTIAL KNOWLEDGE**2.7.A.1**

Iteration is a form of repetition. Iteration statements change the flow of control by repeating a segment of code zero or more times as long as the Boolean expression controlling the loop evaluates to `true`.

2.7.A.2

An *infinite loop* occurs when the Boolean expression in an iterative statement always evaluates to `true`.

2.7.A.3

The loop body of an iterative statement will not execute if the Boolean expression initially evaluates to `false`.

2.7.A.4

Off by one errors occur when the iteration statement loops one time too many or one time too few.

2.7.B.1

A `while` loop is a type of iterative statement. In `while` loops, the Boolean expression is evaluated before each iteration of the loop body, including the first. When the expression evaluates to `true`, the loop body is executed. This continues until the Boolean expression evaluates to `false`, whereupon the iteration terminates.

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 2.8

for Loops

Required Course Content

LEARNING OBJECTIVE

2.8.A

Develop code to represent iterative processes using `for` loops and determine the result of these processes.

ESSENTIAL KNOWLEDGE

2.8.A.1

A `for` loop is a type of iterative statement. There are three parts in a `for` loop header: the initialization, the Boolean expression, and the update.

2.8.A.2

In a `for` loop, the initialization statement is only executed once before the first Boolean expression evaluation. The variable being initialized is referred to as a *loop control variable*. The Boolean expression is evaluated immediately after the loop control variable is initialized and then following each execution of the increment statement until it is `false`. In each iteration, the update is executed after the entire loop body is executed and before the Boolean expression is evaluated again.

2.8.A.3

A `for` loop can be rewritten into an equivalent `while` loop (and vice versa).

TOPIC 2.9

Implementing Selection and Iteration Algorithms

SUGGESTED SKILLS

2.A

Write program code to implement an algorithm.

3.A

Determine the result or output based on statement execution order in an algorithm.

4.A

Describe the behavior of a code segment or program.

Required Course Content

LEARNING OBJECTIVE**2.9.A**

Develop code for standard and original algorithms (without data structures) and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE**2.9.A.1**

There are standard algorithms to:

- identify if an integer is or is not evenly divisible by another integer
- identify the individual digits in an integer
- determine the frequency with which a specific criterion is met
- determine a minimum or maximum value
- compute a sum or average

SUGGESTED SKILLS

2.C

Write program code involving procedural abstractions.

3.C

Determine the result or output based on code that contains procedural abstractions.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

TOPIC 2.10

Implementing String Algorithms

Required Course Content

LEARNING OBJECTIVE**2.10.A**

Develop code for standard and original algorithms that involve strings and determine the result of these algorithms.

ESSENTIAL KNOWLEDGE**2.10.A.1**

There are standard string algorithms to:

- find if one or more substrings have a particular property
- determine the number of substrings that meet specific criteria
- create a new string with the characters reversed

TOPIC 2.11

Nested Iteration

Required Course Content

LEARNING OBJECTIVE**2.11.A**

Develop code to represent nested iterative processes and determine the result of these processes.

ESSENTIAL KNOWLEDGE**2.11.A.1**

Nested iteration statements are iteration statements that appear in the body of another iteration statement. When a loop is nested inside another loop, the inner loop must complete all its iterations before the outer loop can continue to its next iteration.

SUGGESTED SKILLS**2.A**

Write program code to implement an algorithm.

3.D

Explain why a code segment will not compile or work as intended and modify the code to correct the error.

4.B

Describe the initial conditions that must be met for a code segment to work as intended or described.

SUGGESTED SKILLS

4.A

Describe the behavior of a code segment or program.

TOPIC 2.12

Informal Run-Time Analysis

Required Course Content

LEARNING OBJECTIVE**2.12.A**

Calculate statement execution counts and informal run-time comparison of iterative statements.

ESSENTIAL KNOWLEDGE**2.12.A.1**

A *statement execution count* indicates the number of times a statement is executed by the program. Statement execution counts are often calculated informally through tracing and analysis of the iterative statements.